Mini Project Report - 10

Master of Computer Application – General
Semester – III

Sub: Web Technologies

Topic: Library Management System (LMS)
By
Name: Gurudatta Ganapati Bhat
Reg no.: 2511022250081

Faculty Name: VEERA RAGHAV K

Faculty Signature: _____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106

July - 2026

INDEX

INTRODUCTION

A Library Management System (LMS) is a software application used to manage books and control access based on user roles. In this project, Authentication is used to verify the identity of users by checking their username and password. Authorization is used to provide different permissions for different users.
There are three types of users:

- Admin – Can add, update and delete books.
- Faculty – Can view books and take books home.
- Student – Can only view books.

The project is developed using Node.js, Express.js, and tested using Postman with JSON data.

Step 1
Install Node.js from
https://nodejs.org

Step 2
Create Project
mkdir my_LMS
cd my_LMS

Step 3
Initialize Node
npm init -y

Step 4
Install Express
npm install express

Step 5
Create
server.js
Copy the program into the file.

Step 6
Run
node server.js
Output
http://localhost:3000

Postman Installation Steps:

Step 1: Download Postman
- Open your browser.
- Visit https://www.postman.com/downloads/
- Click Download for Windows (or your operating system).

Step 2: Install Postman
- Open the downloaded .exe file.
- Click Next (if prompted).
- Accept the installation process.
- Wait for the installation to complete.

Step 3: Launch Postman
- Open Postman from the Start Menu or Desktop shortcut.

Step 4: Sign In (Optional)
- Sign in using your Google account or create a Postman account.
- You can also click Skip and go to the workspace to use it without signing in.

Step 5: Create a New Request
- Click New → HTTP Request.
- Select the HTTP method (GET, POST, PUT, DELETE).
- Enter the API URL (e.g., http://localhost:3000/login).

Step 6: Send JSON Data
- Click the Body tab.
- Select raw.
- Choose JSON from the dropdown.
- Enter the JSON request body.

input code :

```
const express = require("express");
const app = express();

app.use(express.json());

let currentRole = "";

// Sample Books
let books = [
  { id: 1, title: "Java Programming", author: "James Gosling" },
  { id: 2, title: "Python Basics", author: "Guido van Rossum" },
  { id: 3, title: "Web Technologies", author: "HTML CSS JS" }
];

// ---------------- LOGIN ----------------

app.post("/login", (req, res) => {
```

```
  const { username, password } = req.body;

  if (username === "admin" && password === "123") {
    currentRole = "admin";
    res.json({
      message: "Admin Login Successful"
    });

  } else if (username === "faculty" && password === "123") {
    currentRole = "faculty";
    res.json({
      message: "Faculty Login Successful"
    });

  } else if (username === "student" && password === "123") {
    currentRole = "student";
    res.json({
      message: "Student Login Successful"
    });

  } else {
    res.json({
      message: "Invalid Credentials"
    });
  }

});

// ---------- VIEW BOOKS ----------

app.get("/books", (req, res) => {

  if (
    currentRole === "admin" ||
    currentRole === "faculty" ||
    currentRole === "student"
  ) {
    res.json(books);
  } else {
    res.json({
      message: "Please Login First"
    });
  }

});

// ---------- ADD BOOK ----------

app.post("/add", (req, res) => {

  if (currentRole === "admin") {

    books.push(req.body);

    res.json({
```

```
      message: "Book Added Successfully",
      books
    });

  } else {
    res.json({
      message: "Access Denied"
    });
  }

});

// ---------------- UPDATE BOOK ----------------

app.put("/update/:id", (req, res) => {

  if (currentRole !== "admin") {
    return res.json({
      message: "Access Denied"
    });
  }

  const id = parseInt(req.params.id);

  const book = books.find(b => b.id === id);

  if (!book) {
    return res.json({
      message: "Book Not Found"
    });
  }

  book.title = req.body.title;
  book.author = req.body.author;

  res.json({
    message: "Book Updated Successfully",
    book
  });

});

// ---------------- DELETE BOOK ----------------

app.delete("/delete/:id", (req, res) => {

  if (currentRole !== "admin") {
    return res.json({
      message: "Access Denied"
    });
  }

  const id = parseInt(req.params.id);

  books = books.filter(book => book.id !== id);
```

```
  res.json({
    message: "Book Deleted Successfully",
    books
  });

});

// ---------------- TAKE HOME BOOK ----------------

app.post("/takehome/:id", (req, res) => {

  if (currentRole === "faculty") {

    const id = parseInt(req.params.id);

    const book = books.find(b => b.id === id);

    if (!book) {
      return res.json({
        message: "Book Not Found"
      });
    }

    res.json({
      message: "Book Issued Successfully",
      book
    });

  } else {
    res.json({
      message: "Only Faculty Can Take Books Home"
    });
  }

});

// ---------------- START SERVER ----------------

app.listen(3000, () => {
  console.log("http://localhost:3000");
});
```

Output:

Postman Testing

Admin Login

POST
http://localhost:3000/login

JSON
```
{
   "username":"admin",
   "password":"123"
}
```

Output
```
{
   "message":"Admin Login Successful"
}
```

Faculty Login
```
{
   "username":"faculty",
   "password":"123"
}
```

Output
```
{
   "message":"Faculty Login Successful"
}
```

Student Login
```
{
   "username":"student",
   "password":"123"
}
```

Output
```
{
```

```
   "message":"Student Login Successful"
}
```


View Books

GET
http://localhost:3000/books

Output


```
[
 {
  "id":1,
  "title":"Java Programming",
  "author":"James Gosling"
 },
 {
  "id":2,
  "title":"Python Basics",
  "author":"Guido van Rossum"
 }
]
```


Add Book:

POST
http://localhost:3000/add
Input


```
{
   "id":4,
   "title":"Node JS",
   "author":"Ryan Dahl"
}
```


Output


```
{
  "message":"Book Added Successfully"
}
```


Update Book

PUT
http://localhost:3000/update/2

Input

```
{
   "title":"Python Advanced",
   "author":"Guido"
}
```

Output

```
{
  "message":"Book Updated Successfully"
}
```

Delete Book

DELETE
http://localhost:3000/delete/3

Output

```
{
  "message":"Book Deleted Successfully"
}
```

Faculty Take Home Book
POST
http://localhost:3000/takehome/1

Output

```
{
  "message":"Book Issued Successfully"
}
```

CONCLUSION

The Library Management System successfully demonstrates the concepts of Authentication and Authorization using Node.js and Express.js. Authentication verifies the identity of users, while Authorization restricts access based on user roles. The project enables the Admin to manage books, allows the Faculty to view and borrow books, and permits Students to view books only. The application was tested successfully using Postman with JSON requests, making it a simple and effective example of role-based access control in web applications